Introduction to Programming

Mid Semester Exam (set A)

ROLL NO. _____

INSTRUCTIONS PAGE

1. Please fill your roll number correctly and legibly in the space above. Write your full roll number including department code etc.
2. Do NOT turn over this page till the start of exam is announced (6pm). Once exam starts you may turn over and read the questions overleaf.
3. Please ensure you are carrying only a pen/pencil and your ID card with you.
4. Do NOT carry any electronic devices like mobiles, watches etc. In case invigilators find any device with you, or any loose sheets/notes, you will be awarded FR, even if you were not intending to use it.
5. Please write your answers in this question paper itself. Please write legibly and within the space provided. Submit the filled in question paper at end of exam to the invigilator (9pm).
6. In the unlikely situation where you don't have space in the question paper, you may write on a blank sheet and submit it. Do not forget to write your roll number on such a page. Staple it along with this question paper.
7. You may use blank sheets given to you for rough work, which should also be submitted back to us. Staple it along with this question paper. Do not forget to write your roll number on such a page. Unidentified rough sheets found near you may make you a suspect for copying! Please be careful.
8. Any attempt to copy/cheat etc. will straight away lead to an FR grade in the course.
9. Total marks is 82, but it will be normalized to 15% as announced.
10. After finishing your exam at 9pm, stop writing. Do not write anything beyond 9pm. Leave your filled question paper as it is on your desk and move away immediately towards the big screen. The invigilators will collect your sheets, do a tally. After they announce, you can leave the exam hall.

1 Multiple Choice Questions

Note: Encircle/tick the correct option from A,B,C,D in the following. [2 marks× 10=20 marks]

1. What will be the output of the following C code ?
   A. 1 B. 2 C. 3 D. 4

   Solution: (C) Reason: 1 + 0 + 1 + 0 + 1 + 0

```c
#include <stdio.h>

int foo(int arr[], int n) {
    if (n == 0)
        return 0;
    return ((arr[n-1] == 0) ? 1 : 0) + foo(arr, n-1);
}

int main() {
    int arr[] = {0, 1, 0, 3, 0};
    int n = 5;
    printf("%d", foo(arr, n));
    return 0;
}
```

2. What will be the output of the following C code ?
   A. 20 30 40 50 10 B. 20 30 40 50 50 C. 50 50 50 50 50 D. Undefined Behaviour

   Solution: None of the choices Reason: Ans should be 20 30 40 50 20 which is not given in options. Because, at i=0, index 0 occupies 20 (the value at index 1), and when variable i becomes 4, at that time it will fetch the value of index 0, which is 20, so the last term should be 20.

```c
#include <stdio.h>

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = 5;
    for (int i = 0; i < n; i++) {
        arr[i] = arr[(i+1) % n];
    }
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

3. What will be the output of the following C code ?
   A. 9 B. 16 C. 25 D. 36

   Solution: (C) Reason: The value of a is 3, which is passed to fun2. fun2 is calling fun1 with value 3+2=5, 5. fun1 return 5x5=25, is the ans.

```c
#include <stdio.h>

int fun1(int x) {
    return x * x;
}

int fun2(int y) {
    return fun1(y + 2);
}

int main() {
    int a = 3;
    int result = fun2(a);
    printf("%d", result);
    return 0;
}
```

4. What will be the output of the following C code ?
   A. 2 B. 4 C. 8 D. 16

   Solution: (C) Reason: Because fun will call recursively 3 times, and the corresponding values will be 2x2x2x1=8

```c
#include <stdio.h>

int fun(int n) {
    if (n == 1)
        return 1;
    else
        return 2 * fun(n - 1);
}

int main() {
    int result = fun(4);
    printf("%d", result);
    return 0;
}
```

5. What will be the output of the following C code ?
   A. 16 B. 9 C. 24 D. 25

   Solution: (A). Reason: Because the return value will be n + foo(n-2) when n is odd, and terminates when n<=0 with the return value a 0. Hence the final return value will be 7+5+3+1=16

```c

#include <stdio.h>

int foo(int n) {
    if (n <= 0) return 0;
    if (n % 2 != 0) return n + foo(n - 2);
    return foo(n - 1);
}

int main() {
    int result = foo(8);
    printf("%d", result);
    return 0;
}
```

6. Consider the following C function.

```c

void swap (int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}
```

   In order to exchange the values of two variables, x and y (i.e., after executing the statement, x must contain the original value in y and vice-versa):
   A. call swap (x,y)
   B. call swap (&x, &y)
   C. call swap (y, x)
   D. None of the above

   Solution: (D). Reason: None of the above. Because the function arguments are normal variables and not the pointer. Hence we can not swap the value of x and y.

7. Which one of the following will happen when the function convert, defined below, is called with any positive integer n as an argument?

   A. It will print the binary representation of n in the reverse order and terminate
   B. It will print the binary representation of n but will not terminate

C. It will print the binary representation of n and terminate
D. It will not print anything and will not terminate

Solution: (C). Reason: It will print the binary representation of n and terminate. However, one extra leading zero will be there.

```
void convert(int n){
    if(n==0)
         printf("%d",n);
    else {
         convert(n/2);
         printf("%d",n%2);
    }
}
```

8. The function Trial, defined below, if called with three distinct numbers as its arguments, returns the:
   A. maximum of a, b, and c
   B. minimum of a, b and c
   C. one of a,b,c, which is neither the maximum, nor the minimum
   D. none of the above

   Solution: (C). Reason: One of a,b,c, which is neither the maximum nor the minimum.

```
int Trial (int a, int b, int c)
{
    if ((a > = b) && (c < b)) return b;
    else if (a > = b) return Trial (a,c,b);
    else return Trial (b,a,c);
}
```

9. How many times will the following loop execute?
   A. Forever B. Never C. 2 D. 1

   Solution: (C). Reason: Because when the j'e value becomes 0, and we again subtract 1 from it, an integer underflow will occur, and j will take the highest value, which will be greater than 10.

```
for(unsigned int j = 1; j <= 10; j = j-1)
```

10. What is the output of the following C code?
    A. 52 B. 7654321 C. 7777777 D. Infinite number of 7

    Solution: (D). Reason: Infinite number of 7. Every time, when we call r(), num is initialized with 7, and then it return num using post decrement operation.

```
#include <stdio.h>
int r(){
    int num=7;
    return num--;
}
int main(){
    for (r();r();r())
        printf("%d",r());
    return 0;
}
```

2 Write Short Answer / Fill in the blanks

[2 marks× 5=10 marks]

1. Find the syntax error(s) in the following code. Write the statement(s) which is(are) having syntax error in the first blank. Provide correction(s) so that the statement(s) are free of syntax error. Write the corrected statement(s) in the second blank.

```c
#include <stdio.h>
int main() {
    int x = 10;
    if (x > 5) {
        printf("x is greater than 5");
    }
    else
        printf("x is lesser than 5")
    }
    return 0;
}
```

Error(s) identified:
1: line 8 semicolon missing.
2: line9 brace extra or missing brace in line 7.
Correction(s):
Solution-1:
7 else
8 printf("x is lesser than 5");
9 return 0;
10 }
Solution-2:
7 else {
8 printf("x is lesser than 5");
9 }
10 return 0;
11 }

---

2. Find the syntax error(s) in the following code. Write the statement(s) which is(are) having syntax error in the first blank. Provide correction(s) so that the statement(s) are free of syntax error. Write the corrected statement(s) in the second blank.

```c
#include <stdio.h>

int main() {
    int num = 1;
    int const = 1;
    switch(num) {
        case 1:
            printf("Number is 1.\n");
            const = 2;
            break;
        case 2:
            printf("Number is 2.\n");
            const = 3;
        default
            printf("Number is not 1 or 2.\n");
    }

    return 0;
}
```

Error(s) identified:
1: line 5, 9, 13, we can't use const as a variable name because it is a keyword.
2: line 14, a colon is missing after default.
Correction(s):
Fix-1: line 5, 9, 13, replace const with some other non-keyword name such as aConst.
fix-2: line 14, add a colon after default.

3. Find the syntax error(s) in the following code. Write the statement(s) which is(are) having syntax error in the first blank. Provide correction(s) so that the statement(s) are free of syntax error. Write the

corrected statement(s) in the second blank.

```c
#include <stdio.h>

void find_average(int arr[], int size) {
    int sum = 0;
    float average;

    for (int i = 0; i <= n; i++) {
        sum += arr[i];
    }

    average = sum / size;

    printf("The average is: %f\n", average);
}

int main() {
    int n = 5;
    int array[5] = {1, 2, 3, 4, 5};

    find_average(array, n);

    return 0;
}
```

Error(s) identified:
1: line 7, in for loop, code is using n for condition checking, which is not defined in the function find average.
Correction(s):
Fix-1: line 7, replace n with size.

4. Fill in the blanks (A), (B), (C) in the following C code snippet, so that the intended output will be correctly printed:

```c
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int sum = 0;

    for (int i = 0;_____(A)_____; i++) {
        sum += arr[i];
    }

    printf("Sum of array elements is:_____(B)_____\n", _____(C)_____);
    return 0;
}
```

Blanks to fill:
(A) **Solution: i<5, Comment: Multiple answers are possible: for e.g., i<=4 etc.**
(B) **Solution: %d**
(C) **Solution: sum**

5. Fill in the blanks (A), (B), (C), (D) in the following C code snippet, so that the intended output will be correctly printed:

```c
#include <stdio.h>

_____(A)_____ print_even_numbers(_____(B)_____) {
    for (int i = 0; i <= n; i++) {
        if (_____(C)_____) {
            printf("%d ", i);
        }
    }
```

```
}

int main() {
    int limit = 8;
    print_even_numbers(limit);
    _____(D)_____
}
```

Blanks to fill:
(A) **Solution: void, Comment: Multiple answers are possible: for e.g., int or any datatype etc.**
(B) **Solution: int n, Comment: Multiple answers are possible: for e.g., double n or any appropriate datatype etc. But variable name must be n.**
(C) **Solution: i%2==0, Comment: Multiple answers are possible: for e.g., i%2!=1 etc. But they must not use for example bit-wise and, because it is not taught yet.**
(D) **Solution: return 0;, Comment: Multiple answers possible. For e.g., printf("\n");**

3 Numerical Questions

[2 marks × 5= 10 marks]

1. Consider the following function:

```
void do_something(int n, int arr[n]) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;  //+1 was missing I added now. Some students have already solved it before we announced this correction. So they can write answer based on the original value.
            }
        }
    }
}
```

What will be the output when we pass `arr` and `n` to the above function where, `arr[] = {5, 3, 8, 4, 2};` and `n = 5;` ? Provide a short justification for your answer.

Output: **: answer could be {2, 3, 4, 5, 8} if answered after correction else {5, 3, 8, 4, 2}. Both will work**
Justification: **: Function is implementing bubble sort based on correction. Without correction, it is simply copying value of index j at index j itself, so it will not perform any modification to the array.**

2. What will be the output for the following code? Briefly justify your answer.

```
int arr[7] = {11, 2, 23, 4, 6, 1, 5};
int x = 7;
printf("Fourth element: %d\n", arr[x]);
```

Output: **: output can be a garbage/nondeterministic value**
Justification:**: During the printing, we access the value of index 7. However array size is itself 7, and the maximum index could be 6. Hence, at index 7, there could be any value.**

3. Complete the following code snippet for finding the second largest number in the given array:

```
#include <stdio.h>

int secondLargest(int arr[], int n) {
    // Write the logic here for finding second largest number
    //logic started
    int largest=arr[0], secondLargest=arr[1], temp;
```

```
    if(largest < secondLargest){
        temp = secondLargest;
        secondLargest = largest;
        largest = temp;
    }
    for (int i = 2; i < n; i++) {
        if (arr[i] > largest) {
            secondLargest = largest;
            largest = arr[i];
        } else if (arr[i] > secondLargest && arr[i] != largest) {
            secondLargest = arr[i];
        }
    }
    //logic ended
    ------------------------...
    return secondLargest;
}

int main() {
    int arr[] = {2, 3, 1, 5, 4};
    printf("Second Largest: %d\n", secondLargest(arr, 5));
    return 0;
}
```

Comments: The answer filled above is correct. However, it is not the only correct answer. Some students complained that function name and variable name are same. But gcc is not showing any syntax error.

4. Analyze the following recursive function for calculating the factorial of a number:

```
unsigned long long factorial(int n) {
    return n * factorial(n - 1);
}
```

What will happen when you pass 5 as a parameter to the above function? Will it return the correct factorial of 5 or not? If it does not, what changes would you make to fix it?

Answer:
Solution: The function will not return, it will execute infinitely. To make it work, fix is as follows:

```
unsigned long long factorial(int n) {
    if(n==0 || n==1)
        return 1;
    return n * factorial(n - 1);
}
```

Comments: Again multiple correct answers are possible. for example, if(n==0) return 1; also works.

5. Write a **recursive function** that calculates the sum of elements in an array. The function should take two parameters: the array and its size.

   The function should have the following prototype:

```
int arraySum(const int arr[], int size);
```

Note: There is no need to include the full executable code with the `main()` function; just provide the definition of the recursive function.

Answer:**Answer is:**

```
int arraySum(const int arr[], int size) {
    if(size==0)
        return 0;
    return arr[size-1] + arraySum(arr, size - 1);
}
```

Comment: Again multiple correct answers are possible. for example, if(size==1) return arr[0]; also works.

4 Multiple-Answer Choice Questions

Note: Encircle/tick ALL the correct options in the following. Marks may be awarded only if all the correct are encircled/ticked and none of the wrong options are encircled/ticked.

1. Given below is a code snippet and an associated bar plot (see Figure 1), indicating the number of times variables are accessed between lines 4-11 of the code, across the whole execution of the code. The x-axis entries in the bar plot shown here exactly correspond to a single line number in the code. The corresponding y-axis entry for, say **B1**, gives the number of accesses in line number represented by **B1**. Based on the code shown, which of the following statements are true? **Note**: Consider each element of an array also as an individual variable.

   Solution: D

```
1  #include <stdio.h>
2
3  int main() {
4      double a, b;
5      scanf("%lf %lf", &a, &b);
6      int array[5] = {64, 108, 2, 5, 0};
7      for(int index = 0; index < 5; index++) {
8          if((2+3) == 5) {;}
9          for(int temp = index + 1; temp < 4; temp++)
10         array[temp] += array[index];
11     }
12     return 0;
13 }
```

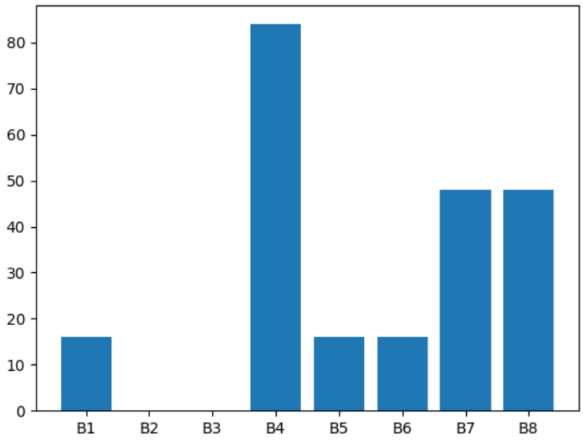

Figure 1: X-axis: line number(s). Y-axis: Number of memory accesses

   A. **B1** could refer to any of lines 4, 5 or 6.
   B. **B8** could refer to any of lines 7 or 10.
   C. The above code gives runtime error after executing line 9 when run.
   D. Line 8 incurs 0 accesses to any variables.
   E. (**B1**, **B2**, **B3**, **B4**, **B5**, **B6**, **B7**, **B8**) = (4, 11, 6, 9, 8, 7, 5, 10).
   F. (**B1**, **B2**, **B3**, **B4**, **B5**, **B6**, **B7**, **B8**) = (5, 11, 7, 10, 4, 6, 9, 10).
   G. (**B1**, **B2**, **B3**, **B4**, **B5**, **B6**, **B7**, **B8**) = (4, 11, 8, 9, 5, 6, 7, 10).
   H. (**B1**, **B2**, **B3**, **B4**, **B5**, **B6**, **B7**, **B8**) = (6, 5, 8, 11, 10, 7, 4, 9).

[6 marks]

2. Select all the **false** statements:
   Solution: B, C, D, E

   A. A static variable will be initialized only once.
   B. One can define two functions in the same C file with identical function names but different defini-

tions, without facing compile-time errors.
C. The `main` function can never take any arguments.
D. By definition, recursive functions always call themselves more than once, when run on any input.
E. `printf` function always returns 0 on successful execution and -1 otherwise.

[2 marks]

3. Select all the **false** statements:
Solution: A, B
A. For any function defined in C, only as many arguments as one more than the number of commas present between the arguments in the function definition can be passed to it.
B. Header files are pre-compiled binary codes that get appended to the binary code of code written by a programmer, before the linking stage.
C. 'if', 'while' and 'for' statements are conditional execution statements.
D. By default, arrays are passed by reference in C language.

[2 marks]

4. Given below are pairs of code snippets, each of which prints some output to stdout. Tick all options where both codes give the same output on any valid input.
Solution: C, D, E
A.

```c
#include <stdio.h>

int main() {

    int a, b;
    scanf("%d %d", &a, &b);
    if(b>a) a = b;
    printf("%d", a);
    return 0;
}
```

```c
#include <stdio.h>
#include <math.h>

int main() {

    int a, b;
    scanf("%d %d", &a, &b);
    printf("%d", ((a+b) + abs(a-b))/2);
    return 0;
}
```

B.

```c
#include <stdio.h>
#include <math.h>

int main() {

    int N;
    scanf("%d", &N);

    // Print row-wise
    for(int row = 0; row < N; row++) {
        for(int col = 0; col < N; col++) {
            char ch = ((col % 2) == 0) ? '*' : '$';
            printf("%c", ch);
        }
        printf("\n"); // Next row
    }

    return 0;
```

```
}
```

,

```
#include <stdio.h>
#include <math.h>

int main() {

    int N;
    scanf("%d", &N);

    // Print column-wise
    for(int col = 0; col < N; col++) {
        char ch = ((col % 2) == 0) ? '*' : '$';
        for(int row = 0; row < N; row++) {
            printf("%c\n", ch);
        }
        printf(" "); // Next column
    }

    return 0;
}
```

C.

```
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N);
    return 0;
}
```

,

```
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N);
    if(printf(""))printf("%d", N);
    return 0;
}
```

D.

```
#include <stdio.h>

int max(int a, int b) {
    return ((a>b) ? a : b);
}

int min(int a, int b) {
    return ((a<b) ? a : b);
}

int main() {
    int N;
    scanf("%d", &N); // N <= 1000
    int arr[1000];

    for(int index = 0; index < N; index++) scanf("%d", &arr[index]);
    int maximum = max(arr[0], 5), minimum = min(arr[0], 5);

    for(int index = 0; index < N; index++) {
        if(arr[index] >= 5)
            minimum = min(arr[index], minimum);
            maximum = max(arr[index], maximum);
    }
```

```
    printf("%d %d", maximum, minimum);

    return 0;
}
```

,

```
#include <stdio.h>

int max(int a, int b) {
    return ((a>b)*a) + ((b>=a)*b);
}

int min(int a, int b) {
    return ((a<b)*a) + ((b<=a)*b);
}

int main() {
    int N;
    scanf("%d", &N); // N <= 1000
    int arr[1000];

    for(int index = 0; index < N; index++) scanf("%d", &arr[index]);
    int maximum = max(arr[0], 5), minimum = min(arr[0], 5);

    for(int index = 0; index < N; index++) {
        if(arr[index] >= 5) {
            maximum = max(arr[index], maximum);
            minimum = min(arr[index], minimum);
        }
    }

    printf("%d %d", maximum, minimum);

    return 0;
}
```

E.

```
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N); // N <= 1000
    int arr[1000];

    for(int index = 0; index < N; index++) scanf("%d", &arr[index]);

    for(int index = 0; index < N; index++) {
        for(int nextIndex = index + 1; nextIndex < N; nextIndex++) {
            if(arr[index] > arr[nextIndex]) {
                int temp = arr[index];
                arr[index] = arr[nextIndex];
                arr[nextIndex] = temp;
            }
        }
    }

    printf("%d", arr[N-1]);

    return 0;
}
```

,

```
#include <stdio.h>

int main() {
    int N;
```

```
    scanf("%d", &N); // N <= 1000
    int arr[1000];

    for(int index = 0; index < N; index++) scanf("%d", &arr[index]);

    int max = arr[0], min = arr[0];
    for(int index = 1; index < N; index++) {
        if(arr[index] > max)
            max = arr[index];
    }

    printf("%d", max);

    return 0;
}
```

[6 marks]

5. Select all statements that are **false** over the following code. Assume valid integers are input from the keyboard in the scanf call:

```
#include <stdio.h>

int main() {
    int a = 0, b = 1;
    if(a > b ? 1 : 0) {
    printf("Hello!\n");
    } else {
    printf("%d", scanf("%dd%d", &a, &b));
    }
    return 0;
}
```

Solution: B, D, E

A. The code when run will never output the string: Hello!.

B. The code when run will halt after taking 1 number as input.

C. When run on integer inputs, the last character output by the code would be 1.

D. The code will throw a compile time error as there is an extra 'd' in the middle of the string input as the first argument to scanf().

E. The last character output by the code would be -1, as the arguments passed to the second call of scanf() are not correct, which would cause the value returned by scanf() to be -1.

[2 marks]

6. Described below here is a task: There is a long strip of land represented by 1-D array $\mathcal{F}$ of size N, where the i^{th} entry in the array corresponds to the i^{th} plot in the land. Some crops need to be planted over this land. However crops can only be planted in one single continuous subsection of this land. Furthermore each cell has a fertility level, which is represented by a fertility score in the array. The crop can be planted in a subsection of the plots only if the total sum of fertility scores in that section is above a threshold T. Given the number of plots N, array of fertility scores $\mathcal{F}$, and the threshold T, output the length of the longest subsection that can be used to plant the crops. Incase no subsection of the plots can be used to plant the crops, report: "No subsection available."

   (Assume that $1 \leq N, T \leq 1e5$ and $-1e9 \leq \mathcal{F}[i] \leq 1e9,\ \forall\ 1 \leq i \leq N$.)

   Which of the following codes output the correct answer to this task on any valid input? Assume that all the codes shown below take the input correctly.

   Solution: B, D, E

   A.

```
#include <stdio.h>

int main() {

    int N, T;
```

```
    int fertility[100000];

    scanf("%d %d", &N, &T);
    for(int index = 0; index < N; index++) scanf("%d", &fertility[index]);
    int answer = -1;

    for(int leftIndex = 0; leftIndex < N; leftIndex++) {
        for(int rightIndex = leftIndex; rightIndex < N; rightIndex++) {
            int sum = 0;
            for(int currIndex = leftIndex; currIndex <= rightIndex; currIndex++) {
                sum += fertility[currIndex];
            }
            if(sum >= T) {
                if ((rightIndex - leftIndex) + 1 > answer) {
                    answer = rightIndex - leftIndex;
                }
            }
        }
    }

    answer++;

    if(answer == -1) {
        printf("No subsection available.");
    } else {
        printf("%d", answer);
    }


    return 0;
}
```

B.

```
#include <stdio.h>

int main() {

    int N, T;
    int fertility[100000];

    scanf("%d %d", &N, &T);
    for(int index = 0; index < N; index++) scanf("%d", &fertility[index]);
    int answer = -1;

    for(int leftIndex = 0; leftIndex < N; leftIndex++) {
        for(int rightIndex = leftIndex; rightIndex < N; rightIndex++) {
            int sum = 0;
            for(int currIndex = leftIndex; currIndex <= rightIndex; currIndex++) {
                sum += fertility[currIndex];
            }
            if(sum >= T) {
                if ((rightIndex - leftIndex) + 1 > answer) {
                    answer = (rightIndex - leftIndex)+1;
                }
            }
        }
    }

    if(answer == -1) {
        printf("No subsection available.");
    } else {
        printf("%d", answer);
    }


    return 0;
}
```

C.

```c
#include <stdio.h>

int main() {

    int N, T;
    int fertility[100000];

    scanf("%d %d", &N, &T);
    for(int index = 0; index < N; index++) scanf("%d", &fertility[index]);
    int answer1 = -1;
    for(int index = 1; index < N; index++) fertility[index] += fertility[index-1];

    for(int index = 0; index < N; index++) {
        if(fertility[index] >= T) {
            answer1 = index;
        }
    }

    int answer2 = -1;
    for(int index = N-1; index >= 1; index--) {
        if(fertility[N-1] - fertility[index-1] >= T) {
            answer2 = N-index;
        }
    }

    if(answer2 > answer1) answer1 = answer2;


    if(answer1 == -1) {
        printf("No subsection available.");
    } else {
        printf("%d", answer1);
    }


    return 0;
}
```

D.

```c
#include <stdio.h>

int main() {

    int N, T;
    int fertility[100001];
    fertility[0] = 0;

    scanf("%d %d", &N, &T);
    for(int index = 0; index < N; index++) scanf("%d", &fertility[index]);
    int answer = -1;
    for(int index = 1; index < N; index++) fertility[index] += fertility[index-1];

    for(int baseIndex = 1; baseIndex <= N; baseIndex++) {
        for(int moveIndex = N; moveIndex >= baseIndex; moveIndex--) {
            if(fertility[moveIndex] - fertility[baseIndex-1] >= T) {
                if(answer-1 < moveIndex - baseIndex) {
                    answer = moveIndex - baseIndex + 1;
                    break;
                }
            }
        }
    }

    if(answer == -1) {
        printf("No subsection available.");
    } else {
        printf("%d", answer);
    }

```

```
31
32      return 0;
33  }
```

E.

```
1  #include <stdio.h>
2
3  int main() {
4
5      int N, T;
6      int fertility[100001];
7      fertility[0] = 0;
8
9      scanf("%d %d", &N, &T);
10     for(int index = 0; index < N; index++) scanf("%d", &fertility[index]);
11     int answer = -1;
12     for(int index = 1; index < N; index++) fertility[index] += fertility[index-1];
13     int totalSum = fertility[N-1];
14
15     for(int baseIndex = 1; baseIndex <= N; baseIndex++) {
16         for(int moveIndex = N; moveIndex >= baseIndex; moveIndex--) {
17             int first = fertility[baseIndex-1];
18             int second = fertility[N] - fertility[moveIndex];
19             if(totalSum - first - second >= T) {
20                 if(answer < moveIndex - baseIndex + 1) {
21                     answer = moveIndex - baseIndex + 1;
22                     break;
23                 }
24             }
25         }
26     }
27
28     if(answer == -1) {
29         printf("No subsection available.");
30     } else {
31         printf("%d", answer);
32     }
33
34
35     return 0;
36 }
```

[6 marks]

7. Your friend has written a code which compiles correctly, but faces a segmentation fault after running a few lines of the code. Which of the following are possible options to explore to help your friend debug the code?

   Solution: A

   A. There could be an array declared where addresses declared outside the scope of the array are trying to be accessed.

   B. There could be some intermediate computation in the code which later leads to a divide by 0 style error.

   C. A array with variable size might have been declared in global scope.

   D. May have declared an array with type 'int' but passed values to be stored in it as 'double' which may have directly caused the segmentation fault.

   E. May have accidentally typecast a float variable into char datatype.

[2 marks]

5 Descriptive Coding

Atindra and Sragvi have been tasked with developing a new system to manage the student Gymkhana elections at their institute. This system consists of two interfaces: one for voters and one for candidates. Your task is to implement these interfaces.

Voter Interface

Comment: Multiple correct answers are possible. Below are the one possible solution for each problem.
The voter interface must provide the following functionalities:

1. **View candidate manifesto**: Allow voters to read the manifesto of each candidate.
2. **Cast vote**: Allow voters to cast a vote for a candidate by entering the candidate's ID.
3. **Change vote**: Allow voters to change their vote if they have already voted.

Function Declarations and Definitions

You need to implement the following functions:

```
// Function to view the manifesto of a candidate
void viewManifesto(char manifestos[][][100], int candidateID);
// manifestos[i][j][k] will be the k^th character in the j^th line of the i^th candidates manifesto. Assume, each line has maximum 100 characters.
// candidateID is the index of the candidate whose manifesto needs to be printed.
```

[2 marks]

Write this function's definition here: **Definition:**

```
// Function to view the manifesto of a candidate
void viewManifesto(char manifestos[][][100], int candidateID){
    // manifestos[i][j][k] will be the k^th character in the j^th line of the i^th candidates manifesto. Assume, each line has maximum 100 characters.
    // candidateID is the index of the candidate whose manifesto needs to be printed.

    int lines = 50; // assuming one manifesto has 50 lines (student can assume anything else).

    for(int j=0; j<lines; j++){
        for(int k=0; ((k<100) && (manifestos[candidateID][j][k]!='\0')); k++){
            printf("%c",manifestos[candidateID][j][k]);
        }
        printf("\n");
    }
}
```

```
// Function to cast a vote for a candidate
void castVote(int votes[][], int voterID, int candidateID);
// votes is a binary matrix, whose row sum is unity. votes[i][j] will be 1 iff i^th voter votes for j^th candidate
// candidateID is the index of the candidate whom the voter with index as voterID wishes to vote for.
```

[2 marks]

Write this function's definition here: **Definition:**

```
// Function to cast a vote for a candidate
void castVote(int votes[][], int voterID, int candidateID){
// votes is a binary matrix, whose row sum is unity. votes[i][j] will be 1 iff i^th voter votes for j^th candidate
// candidateID is the index of the candidate whom the voter with index as voterID wishes to vote for.
    votes[voterID][candidateID] = 1;
}
```

```c
// Function to change the vote to a different candidate
void changeVote(int votes[][], int voterID, int prevCandidateID, int newCandidateID);
```

[2 marks]

Write this function's definition here: **Definition:**

```c
// Function to change the vote to a different candidate
void changeVote(int votes[][], int voterID, int prevCandidateID, int newCandidateID){
    votes[voterID][prevCandidateID]=0;
    votes[voterID][newCandidateID]=1;
}
```

Candidate Interface

The candidate interface must provide the following functionalities:

1. **Update manifesto**: Allow candidates to update their manifesto by specifying which line they want to modify.
2. **Display the number of views for the manifesto**: Display the number of views in a visual format using LED-style notation for each digit.

Function Declarations and Definitions

You need to implement the following functions:

```c
// Function to update a specific line in the manifesto
void updateManifesto(char manifestos[][][100], int candidateID, int lineID, char newLine
    []);
// the sentence in lineID needs to be replaced by that in newLine.
```

[2 marks]

Write this function's definition here: **Definition:**

```c
// Function to update a specific line in the manifesto
void updateManifesto(char manifestos[][][100], int candidateID, int lineID, char newLine
    []){
// the sentence in lineID needs to be replaced by that in newLine.
    for(int i=0;i<100; i++){
        manifestos[candidateID][lineID][i] = newLine[i];
        if(newline[i]=='\0')
            break;
    }
}
```

```c
// Function to display the number of views in LED-style notation
void displayViews(int views);
```

An example of how the number 120 would be displayed in LED-style notation is as follows:

```
*     *  *  *   *  *  *
*           *   *     *
*     *  *  *   *     *
*     *        *     *
*     *  *  *   *  *  *
```

[8 marks]

Write this function's definition here: **Definition:**

```c
// Function to display the number of views in LED-style notation
void displayViews(int views){
    int length = 0, num=views;

```

```
    // defining the digits in ascii form using 3D character array
    char numbers[10][5][3] = {
        {
            {'*', '*', '*'},
            {'*', ' ', '*'},
            {'*', ' ', '*'},
            {'*', ' ', '*'},
            {'*', '*', '*'}
        },
        {
            {' ', '*', ' '},
            {' ', '*', ' '},
            {' ', '*', ' '},
            {' ', '*', ' '},
            {' ', '*', ' '}
        },
        {
            {'*', '*', '*'},
            {' ', ' ', '*'},
            {'*', '*', '*'},
            {'*', ' ', ' '},
            {'*', '*', '*'}
        },
        {
            {'*', '*', '*'},
            {' ', ' ', '*'},
            {'*', '*', '*'},
            {' ', ' ', '*'},
            {'*', '*', '*'}
        },
        {
            {'*', ' ', '*'},
            {'*', ' ', '*'},
            {'*', '*', '*'},
            {' ', ' ', '*'},
            {' ', ' ', '*'}
        },
        {
            {'*', '*', '*'},
            {'*', ' ', ' '},
            {'*', '*', '*'},
            {' ', ' ', '*'},
            {'*', '*', '*'}
        },
        {
            {'*', '*', '*'},
            {'*', ' ', ' '},
            {'*', '*', '*'},
            {'*', ' ', '*'},
            {'*', '*', '*'}
        },
        {
            {'*', '*', '*'},
            {' ', ' ', '*'},
            {' ', ' ', '*'},
            {' ', ' ', '*'},
            {' ', ' ', '*'},
        },
        {
            {'*', '*', '*'},
            {'*', ' ', '*'},
            {'*', '*', '*'},
            {'*', ' ', '*'},
            {'*', '*', '*'}
        },
        {
            {'*', '*', '*'},
            {'*', ' ', '*'},
            {'*', '*', '*'},
            {' ', ' ', '*'},
```

```
            {'*', '*', '*'}
        }
    };

    // calculating the length of views (number of digits)
    do{
        length++;
        num /=10;
    }while(num);

    // separating the digits and storing in an array
    int digits[length];
    if(length==1){
        digits[0]=views;
    }
    else{
        int len2=length;
        num=views;
        while(num){
            digits[len2-1] = num%10;
            num /=10;
            len2--;
        }
    }

    for(int i=0;i<5;i++) //outer loop for lines in asci digits
    {
        for(int k=0;k<length;k++) // Length of views
        {
            for(int j=0;j<3;j++)
            {
                printf("%c",numbers[digits[k]][i][j]); //Printing each line based on digit
            }
            printf("  ");
        }
        printf("\n");
    }
}
```